UNIT-I
Introduction to IoT and Sensor Networks

Syllabus: Introduction to Internet of Things (IoT), Characteristics and Applications of IoT, IoT Architecture and Reference Models (IETF, ITU-T), Physical Design of IoT: Devices, Gateways, and Data Centers, Functional Blocks of IoT: Sensing, Actuation, Communication, Enabling Technologies: RFID, Wireless Sensor Networks, Nanotechnology, Basics of Networking and Communication Protocols: MQTT, CoAP, ZigBee, HTTP, Sensor Networks: Types, Topologies, and Protocols Introduction to IoT Security and Privacy Fundamentals.

1.1. Definition

A dynamic global network infrastructure with self-configuring capabilities based on standard and interoperable communication protocols where physical and virtual "things" have identities, physical attributes and virtual personalities and use intelligent interfaces, and are seamlessly integrated into information network, often communicate data associated with users and their environments.

1.1.1 Alternative Definition

A global infrastructure for the information society, enabling advanced services by interconnecting (physical and virtual) things based on existing and evolving interoperable information and communication technologies.

1.2. Characteristics of IoT

1) **Dynamic & Self Adapting**: IoT devices and systems may have the capability to dynamically adapt with the changing contexts and take actions based on their operating conditions, user's context or sensed environment.

   Ex: Surveillance system – can adapt normal (day) and night time modes

2) **Self-Configuring:** allowing a large number of devices to work together to provide certain functionality.

   Ex: smart home lighting system. When introduce a new smart light bulb into the network, it doesn't require manual configuration. Instead, it automatically detects the presence of the smart home hub, connects to the network, obtains a unique IP address, and registers itself with the hub.

3) **Inter Operable Communication Protocols:** support a number of interoperable communication protocols and can communicate with other devices and also with infrastructure. **Ex:** Using an IoT gateway to translate between different smart home protocols, like Zigbee and Bluetooth, allowing devices from various brands to function together.

4) **Unique Identity:** Each IoT device has a unique identity and a unique identifier (IP address). **Ex:** Asset Tracking: Each item or piece of equipment in a logistics company is tagged with an RFID label carrying a unique code, so its location and status can be updated continually in a database.

5) **Integrated into Information Network:** that allow them to communicate and exchange data with other devices and systems.

Ex: Smart Home Appliances: LG and Samsung smart appliances connect to user phones and cloud servers, reporting status, receiving commands, and interacting with digital assistants like Alexa.

1.3. Applications of IoT/ Domain Specific IoT

1) Home Automation:

a) **Smart Lighting:** helps in saving energy by adapting the lighting to the ambient conditions and switching on/off or diming the light when needed.

b) **Smart Appliances:** make the management easier and also provide status information to the users remotely.

c) **Intrusion Detection:** use security cameras and sensors (PIR sensors and door sensors) to detect intrusion and raise alerts. Alerts can be in the form of SMS or email sent to the user.

d) **Smoke/Gas Detectors:** Smoke detectors are installed in homes and buildings to detect smoke that is typically an early sign of fire. Alerts raised by smoke detectors can be in the form of signals to a fire alarm system. Gas detectors can detect the presence of harmful gases such as CO, LPG etc.,

2) Cities:

a) **Smart Parking:** make the search for parking space easier and convenient for drivers. Smart parking are powered by IoT systems that detect the no. of empty parking slots and send information over internet to smart application backends.

b) **Smart Lighting:** for roads, parks and buildings can help in saving energy.

c) **Smart Roads:** Equipped with sensors can provide information on driving condition, travel time estimating and alert in case of poor driving conditions, traffic condition and accidents.

d) **Structural Health Monitoring:** uses a network of sensors to monitor the vibration levels in the

structures such as bridges and buildings.

e) **Surveillance:** The video feeds from surveillance cameras can be aggregated in cloud based scalable storage solution.

f) **Emergency Response:** IoT systems for fire detection, gas and water leakage detection can help in generating alerts and minimizing their effects on the critical infrastructures.

3) Environment:

a) **Weather Monitoring:** Systems collect data from a no. of sensors attached and send the data to cloud-based applications and storage back ends. The data collected in cloud can then be analysed and visualized by cloud-based applications.

b) **Air Pollution Monitoring:** System can monitor emission of harmful gases (CO2, CO, NO, NO2 etc.,) by factories and automobiles using gaseous and meteorological sensors. The collected data can be analysed to make informed decisions on pollutions control approaches.

c) **Noise Pollution Monitoring:** Due to growing urban development, noise levels in cities have increased and even become alarmingly high in some cities. IoT based noise pollution monitoring systems use a no. of noise monitoring systems that are deployed at different places in a city. The data on noise levels from the station is collected on servers or in the cloud. The collected data is then aggregated to generate noise maps.

d) **Forest Fire Detection:** Forest fire can cause damage to natural resources, property and human life. Early detection of forest fire can help in minimizing damage.

e) **River Flood Detection:** River floods can cause damage to natural and human resources and human life. Early warnings of floods can be given by monitoring the water level and flow rate. IoT based river flood monitoring system uses a no. of sensor nodes that monitor the water level and flow rate sensors.

4) Energy:

a) **Smart Grids:** is a data communication network integrated with the electrical grids that collects and analyze data captured in near-real-time about power transmission, distribution and consumption. Smart grid technology provides predictive information and recommendations to utilities, their suppliers, and their customers on how best to manage power. By using IoT based sensing and measurement technologies, the health of equipment and integrity of the grid can be evaluated.

b) **Renewable Energy Systems:** IoT based systems integrated with the transformers at the point

of interconnection measure the electrical variables and how much power is fed into the grid. For wind energy systems, closed-loop controls can be used to regulate the voltage at point of interconnection which coordinate wind turbine outputs and provides power support.

c) **Prognostics:** In systems such as power grids, real-time information is collected using specialized electrical sensors called Phasor Measurement Units (PMUs) at the sub stations. The information received from PMUs must be monitored in real- time for estimating the state of the system and for predicting failures.

5) Retail:

a) **Inventory Management:** IoT systems enable remote monitoring of inventory using data collected by RFID readers.

b) **Smart Payments:** Solutions such as contact-less payments powered by technologies such as Near Field Communication (NFC) and Bluetooth.

c) **Smart Vending Machines:** Sensors in a smart vending machine monitors its operations and send the data to cloud which can be used for predictive maintenance.

6) Logistics:

a) **Route generation & scheduling:** IoT based system backed by cloud can provide first response to the route generation queries and can be scaled up to serve a large transportation network.

b) **Fleet Tracking:** Use GPS to track locations of vehicles in real-time.

c) **Shipment Monitoring:** IoT based shipment monitoring systems use sensors such as temp, humidity, to monitor the conditions and send data to cloud, where it can be analysed to detect food spoilage.

d) **Remote Vehicle Diagnostics:** Systems use on-board IoT devices for collecting data on Vehicle operations (speed, RPM etc.,) and status of various vehicle subsystems.

7) Agriculture:

a) **Smart Irrigation:** Uses soil moisture sensors and live weather data to automatically trigger water pumps only when crops need hydration. Prevents water waste and overwatering, significantly cutting resource costs while maintaining optimal soil conditions.

b) **Green House Control:** Continuously monitors temperature, humidity, and light levels to automatically drive fans, vents, and grow lights. Creates an ideal microclimate for year-round farming, protecting crops from weather extremes while maximizing yields.

8) Industry:

a) **Machine diagnosis and prognosis:** Uses IoT sensors (like vibration, thermal, and current meters) paired with AI to detect equipment anomalies and forecast remaining useful life. Enables predictive maintenance to fix machinery *before* it breaks down, drastically reducing unexpected downtime and costly repairs.

b) **Indoor Air Quality Monitoring:** Continuously measures gas leaks, particulate matter ($PM_{2.5}/PM_{10}$), CO_2, and humidity levels across factory floors using smart environmental sensors. Automatically triggers ventilation systems when pollutants spike, safeguarding worker health and ensuring compliance with safety regulations.

9) Health and Lifestyle:

a) **Health & Fitness Monitoring:** Uses IoT-connected sensors to continuously stream physical activity, vital signs (like heart rate and blood oxygen), and sleep metrics to cloud dashboards. Provides real-time health insights, tracks fitness goals dynamically, and automatically sends immediate alerts to users or doctors if irregular vitals are detected.

b) **Wearable Electronics:** Combines compact hardware (such as smartwatches, rings, and smart garments) with low-power wireless connectivity like Bluetooth Low Energy (BLE) to collect body and location data on the go. Enables seamless hands-free navigation, real-time lifestyle tracking, and instant notifications, integrating effortlessly into daily personal routines.

1.4. Introduction to IoT Architecture

An IoT Reference Architecture provides a standardized framework/blueprint to design, build, and deploy interoperable IoT networks. Because physical IoT devices vary greatly (from resource-constrained microcontrollers to high-power edge gateways), standard organizations like ITU-T and IETF defined reference models to standardize protocol stacks and system design.

1.4.1. ITU-T IoT Reference Model

The International Telecommunication Union - Telecommunication Standardization Sector (ITU-T) defines a standard 4-Layer Reference Model cross-cut by Management and Security capabilities.

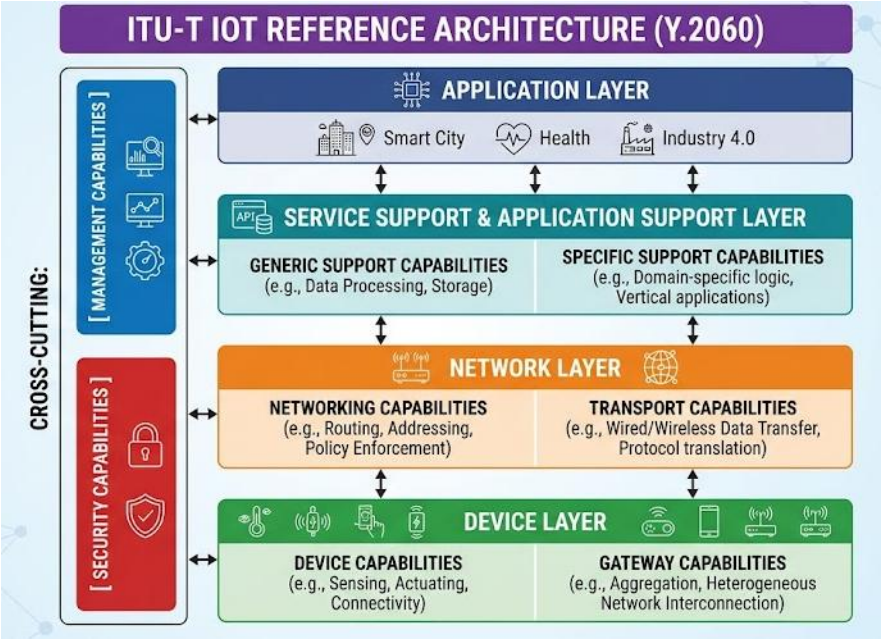


Figure1.1: ITU-T IoT Reference Architecture

Layer Description:

1. Device Layer

Focuses on physical hardware and hardware-to-network interfaces.

- Device Capabilities: Direct or indirect interaction with networks, sleep/wake cycles for energy optimization, and ad-hoc networking.
- Gateway Capabilities: Protocol conversion (e.g., converting ZigBee/Bluetooth data to IP/Ethernet) and bridging multiple network interfaces.

2. Network Layer

Responsible for reliable data transport and routing between devices and central systems.

- Networking Capabilities: Control functions like AAA (Authentication, Authorization, Accounting), mobility management, and network resource allocation.
- Transport Capabilities: Physical transport of application data and control signals across networks (e.g., 4G/5G, Wi-Fi, Ethernet).

3. Service Support & Application Support Layer

Acts as middleware providing essential software services.

- Generic Support Capabilities: Common functions used across multiple IoT applications (e.g., data processing, storage, database management).
- Specific Support Capabilities: Tailored capabilities required for specialized industries (e.g.,

specific telemetry protocols for industrial automation or smart grids).

4. Application Layer

The top layer containing end-user IoT applications (e.g., Smart Agriculture, Smart Cities, Connected Healthcare, Fleet Management).

Cross-Cutting Capabilities (Vertical Blocks):

- Management Capabilities: Device life-cycle management, remote firmware updates (FOTA), performance monitoring, and fault management.
- Security Capabilities: Device authentication, data confidentiality, privacy, and secure communication channels.

1.4.2. IETF IoT Architecture (The IP-Smart Object Model)

The **Internet Engineering Task Force (IETF)** focuses on adapting standard Internet Protocols (IP) for low-power, constrained-resource environments. Instead of reinventing the internet, IETF created specialized protocols at every layer of the standard IP protocol stack.

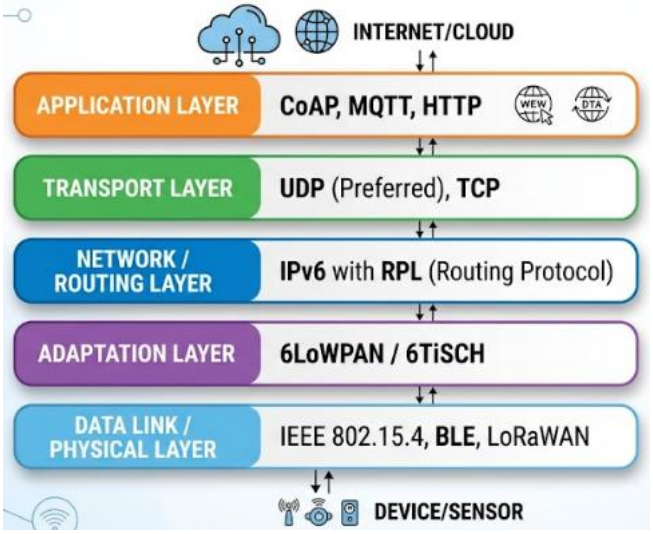


Figure 1.2: IETF IoT Architecture

Layer-by-Layer Protocol Breakdown:

1. **Physical & Data Link Layer:**
    - Handles low-power radio communication on the physical medium.
2. **Adaptation Layer:**
    - *IPv6 over Low-Power Wireless Personal Area Networks.*
    - Adapt IPv6 (128-bit addresses, min payload size of 1280 bytes) to fit into small IEEE 802.15.4 frames (max payload size of 127 bytes) through header compression,

fragmentation, and mesh routing.

3. **Network Layer:**
    - IPv6: Provides unique IP addressing for billions of constrained nodes.
    - RPL (Routing Protocol for Low-Power and Lossy Networks): A distance-vector routing protocol that builds Destination-Oriented Directed Acyclic Graphs (DODAGs) to minimize energy consumption.
4. **Transport Layer (UDP):**
    - Uses UDP (User Datagram Protocol) over TCP because UDP lacks heavy connection setup, handshakes, and retransmission overhead—making it light and energy-efficient for battery-operated nodes.
5. **Application Layer (CoAP / CoRE):**
    - CoAP (Constrained Application Protocol): A specialized RESTful web transfer protocol running over UDP that mimics HTTP (GET, POST, PUT, DELETE) with a small 4-byte header footprint.
    - CoRE (Constrained RESTful Environments): Defines resource discovery mechanisms

1.5. Physical Design Of IoT: Devices, Gateways, Data centres

The Physical Design of IoT refers to the individual components that make up an IoT system — their functions, and the physical structures/technologies they use. The three core building blocks are:

1. IoT Devices/Things
2. IoT Gateways
3. Data Centers/Cloud

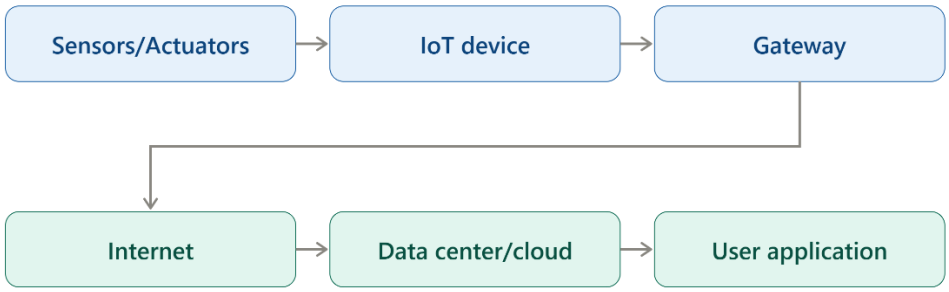


Figure: IoT data flow block diagram

1.5.1. IoT Devices

An IoT device (or "Thing") is a piece of hardware with sensing/actuating, processing, communicating, and (optionally) monitoring/controlling capability.

Generic block diagram components:

Block	Function
Sensors	Sense physical parameters (temperature, humidity, motion, light, etc.) and convert to electrical/digital signals
Actuators	Perform physical actions (motors, relays, switches) based on control signals
Processor/Microcontroller	Local processing, decision-making, data formatting
Communication Module	Sends/receives data (Wi-Fi, Bluetooth, Zigbee, RFID, GSM, LoRa)
I/O Interfaces	Ports for connecting external peripherals
Power Supply	Battery/mains/energy harvesting

1.5.2. IoT Gateways

A Gateway acts as a bridge between the IoT device network (using short-range, low-power protocols) and the external network/Internet (using standard IP-based protocols).

Functions of a Gateway:

- Protocol translation – converts device protocols (Zigbee, BLE, Z-Wave) into IP-based protocols (Wi-Fi, Ethernet)
- Data aggregation & filtering – collects data from multiple devices, removes redundancy
- Local processing/edge analytics – reduces load on cloud, enables faster response
- Security – provides authentication, encryption, firewall functions
- Device management – monitors connected device status

Need of Gateway:

Most IoT devices cannot connect directly to the Internet due to low power/limited protocol stacks — the gateway solves this interoperability problem.

1.5.3. Data Centers / Cloud

The Data Center stores, processes, and analyzes the massive volume of data received from gateways.

Components/Functions:

Component	Role
Servers	Host applications, run analytics/ML models
Storage systems	Store structured/unstructured IoT data (databases, data lakes)
Networking equipment	Ensures connectivity and load balancing
Cloud platforms	Provide scalable compute (AWS IoT, Azure IoT, Google Cloud IoT)
Analytics engine	Processes data for insights, dashboards, alerts

Services offered by data centre/cloud:

- Data storage & backup
- Big data analytics
- Machine learning/AI processing
- Remote device management
- Visualization dashboards for end users

1.6. Functional Blocks of IoT: Sensing, Actuation, Communication

The Functional Blocks of IoT describe the logical/functional units that together enable an IoT system to sense the environment, make decisions, act on it, and exchange data. The standard functional blocks are: Device, Communication, Services, Management, Security, and Application — but the three core operational blocks are:

1. Sensing
2. Actuation
3. Communication

1.6.1. Sensing Block

Responsible for collecting real-time data from the physical environment using sensors. Sensing is the input stage of IoT — it captures data from the physical world into the digital world.

Function of Sensing block

- Detects physical parameters (temperature, humidity, pressure, motion, light, gas, proximity, etc.)
- Converts physical/analog quantities into electrical signals (analog or digital)
- Sends data to processing unit for further action

IoT Sensing Types

Sensing can be divided into four different categories based on the nature of the environment being sensed and the sensors being used;

1. Scalar sensing
2. Multimedia sensing or Vector Sensing
3. Hybrid sensing
4. Virtual sensing

1. Scalar Sensing

Scalar sensing involves the collection of single-value data points from the environment. This type of sensing typically uses traditional sensors that measure a specific physical quantity and output a numerical value.

- **Characteristics:** Focuses on single, quantifiable measurements like temperature, pressure, humidity, light intensity, etc.
- **Sensors Used:** Thermometers, barometers, hygrometers, simple light sensors, etc..
- **Examples:** A temperature sensor reporting the ambient temperature, a pressure sensor measuring tire pressure in a vehicle, or a light sensor detecting the brightness level in a room for automatic lighting control.

2.Vector Sensing

Vector sensing in IoT provides spatial intelligence by measuring quantities that have both magnitude and direction. Vector sensing involves measuring physical phenomena across multiple dimensions (usually X, Y, and Z axes). While a scalar sensor measures "how much" (e.g., temperature), a vector sensor measures "how much" and "in what direction" (e.g., acceleration). This allows IoT devices to determine their orientation, movement, and position in 3D space.

- **Characteristics:** Operates on a coordinate system, Produces multiple data streams per sensor.
- **Sensors Used:** Accelerometers, Gyroscopes, Magnetometers, Inertial Measurement Units (IMUs)
- **Examples:** Autonomous Navigation, Predictive Maintenance, Asset Tracking, Human Activity Recognition, Structural Health, etc.

3. Multimedia Sensing

Multimedia sensing involves the collection of rich, multi-dimensional data, such as images, videos, or audio. This type of sensing aims to capture complex environmental information that cannot be represented by a single numerical value.

- **Characteristics:** Deals with large volumes of data that often require processing and interpretation to extract meaningful information.
- **Sensors Used:** Cameras (for still images and video), microphones, and other sensors capable of capturing visual or auditory data.
- **Examples:** A security camera recording video surveillance, a microphone capturing sound for environmental monitoring, or a drone equipped with a camera for aerial mapping.

4. Hybrid Sensing

Hybrid sensing combines different types of sensing, typically integrating both scalar and multimedia sensing capabilities. This approach provides a more comprehensive understanding of the environment by correlating various data streams.

- **Characteristics:** Integrates diverse sensor types to gather a more complete picture of the sensed environment. Data fusion techniques are often employed to combine and analyze the different data streams.
- **Sensors Used:** A combination of scalar sensors (e.g., temperature, pressure) and multimedia sensors (e.g., cameras, microphones) within a single system or network.
- **Examples:** A smart home system that uses temperature sensors and motion detectors along with security cameras and voice control, or an industrial monitoring system that combines vibration sensors with thermal imaging to detect equipment malfunctions.

5. Virtual Sensing

Virtual sensing, also known as soft sensing or inferred sensing, involves deriving information about a parameter that is not directly measured by a physical sensor. Instead, it uses data from other physical sensors and computational models to estimate or predict the value of the unmeasured parameter.

- **Characteristics:** Relies on data processing and modeling rather than direct physical measurement for a specific parameter. Useful when direct sensing is impractical, too costly, or impossible.

- **Methods Used:** Algorithms, mathematical models, and data fusion techniques applied to data from other physical sensors.
- **Examples:** Estimating the concentration of a certain chemical in a reactor based on temperature and pressure readings and a chemical process model, or predicting road conditions (e.g., ice) based on external temperature and humidity measurements without a direct ice sensor

1.6.2. Actuation Block

Responsible for performing a physical action in response to processed data/commands, converting electrical signals back into physical movement or change. Actuation is the output stage of IoT — it converts digital decisions into real-world physical effects, closing the sense-process-act loop.

Function:

- Receives control signals from processing unit/cloud
- Converts electrical signal into mechanical/physical action
- Enables the IoT system to respond/control the environment (not just observe it)

Types of Actuators:

Type	Example
Motor-based	DC motor, Servo motor, Stepper motor
Relay-based	Switching electrical circuits (on/off)
Solenoid	Valve control
Pneumatic/Hydraulic	Industrial actuators

Examples:

- Smart irrigation → actuator turns water pump ON/OFF based on soil moisture sensor data
- Smart AC → actuator adjusts compressor speed based on temperature sensor reading

1.6.3. Communication Block

Responsible for transferring sensed data from devices to gateway/cloud, and control commands back from cloud/gateway to actuators. Communication is the bridge connecting sensing and actuation blocks to the gateway, cloud, and user applications — enabling remote monitoring and control.

Function:

- Enables device-to-device (D2D), device-to-gateway (D2G), and device-to-cloud (D2C) connectivity
- Uses various wireless/wired protocols depending on range, power, and bandwidth needs

Common Communication Protocols:

Protocol	Range	Power	Data Rate	Use Case
Bluetooth/BLE	~10-100m	Low	Medium	Wearables
Zigbee	~10-100m	Very Low	Low	Home automation
Wi-Fi	~50m	Medium	High	Home/office IoT
RFID/NFC	Few cm-meters	Very Low	Low	Identification/tracking
LoRaWAN	Km range	Very Low	Low	Smart agriculture, smart city
GSM/4G/5G	Long range	High	High	Remote monitoring
Ethernet	Wired	—	High	Gateway/backbone connectivity

5. How the Three Blocks Work Together

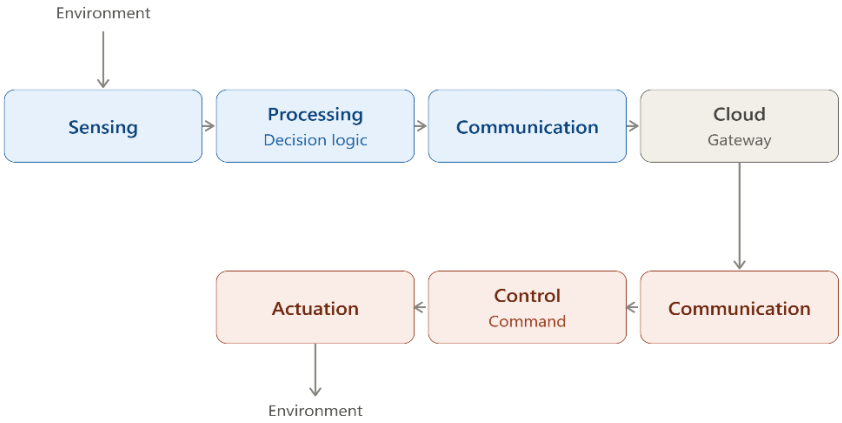


Figure 1.4: IoT sensing and actuation loop

Example (Smart Home Fire Alarm):

1. Sensing → Smoke sensor detects smoke
2. Communication → Data sent via Wi-Fi to cloud/gateway
3. Processing → Cloud analyzes and triggers alert
4. Communication → Command sent back to device
5. Actuation → Alarm buzzer turns ON, sprinkler activates

1.7. Enabling Technologies of IoT: RFID, WSN, Nano Technology

Enabling Technologies refers to the underlying hardware and software technologies that make the Internet of Things possible.

1.7.1. RFID (Radio Frequency Identification)

Wireless technology using radio waves to identify/track tags attached to objects, without line of sight.

Components:

- Tag (Transponder) – stores object's unique ID
- Reader (Transceiver) – sends/receives radio signals
- Antenna – enables communication between tag & reader
- Backend system – processes/stores data

Types of Tags:

Type	Power	Range	Cost	Use
Passive	No battery, powered by reader	Short	Low	Retail, access cards
Active	Own battery	Long (~100m)	High	Toll systems, asset tracking
Semi-passive	Battery for chip only	Medium	Medium	Cold-chain monitoring

Working: Reader emits radio waves → tag gets energized/activated → tag sends back stored data → reader converts to digital data → sent to backend.

Frequency bands: LF (125–134 KHz), HF (13.56 MHz – NFC), UHF (860–960 MHz – logistics), Microwave (2.45 GHz – tolling)

Applications: Supply chain, access control, healthcare tracking, toll collection (FASTag), livestock tracking, contactless payments

Pros: No line of sight needed, fast bulk reading, durable

Cons: Interference from metal/liquid, tag cost, privacy concerns, limited passive-tag range

1.7.2. Wireless Sensor Networks (WSN)

Network of spatially distributed autonomous sensor nodes that monitor physical conditions (temperature, motion, pressure, etc.) and relay data to a base station.

Sensor Node Architecture (draw this in exam):

- Sensing unit – sensor + ADC
- Processing unit – microcontroller + memory
- Transceiver unit – wireless transmit/receive

- Power unit – battery (+ often energy harvesting)

Network flow: Sensor Node → Cluster Head → Sink/Base Station → Gateway → Internet/User

Characteristics:

- Large-scale deployment (hundreds–thousands of nodes)
- Self-organizing/self-healing
- Limited power, memory, computation
- Dynamic topology
- Data-centric routing (not address-centric)

Topologies: Star (direct to base station), Mesh (multi-hop relay), Tree/Hybrid.

Routing Protocols: LEACH (cluster-based), Directed Diffusion (data-centric), SPIN (negotiation-based).

Applications: Environmental monitoring, military surveillance, healthcare monitoring, industrial automation, smart agriculture, disaster management.

Challenges: Energy constraints, limited bandwidth/processing, security vulnerabilities, scalability, fault tolerance.

1.7.3. Nanotechnology

Science/engineering of materials and devices at nanoscale (1–100 nm), enabling extremely small sensors and actuators for IoT.

Role in IoT: Enables miniaturization → leads to Nano-sensors, Nano-nodes → forms Wireless Nano-Sensor Networks (WNSN), an extension of WSN at nano-scale.

Nano-Components:

- Nano-node – smallest unit, limited computation
- Nano-router – aggregates & controls nano-nodes
- Nano-micro interface device – bridges nano-network to classical network
- Gateway – connects to Internet

Applications:

- Healthcare – drug delivery, in-body diagnostics
- Environmental – detecting toxic gases at molecular level
- Military – chemical/biological threat detection
- Industrial – structural health monitoring
- Agriculture – soil quality, pesticide detection

Advantages: Extremely small size, low power, high sensitivity/precision, enables smart materials.

Challenges: Very short communication range, need for new paradigms (e.g., THz band communication), manufacturing complexity, standardization still evolving.

1.8. Basics of Networks:

1.8.1. Network

A network is a collection of interconnected devices (computers, servers, IoT nodes, etc.) that can exchange data using common rules called protocols. Uses of network are to Share resources (files, printers, internet), Enable communication (email, video calls, APIs), Support distributed applications (cloud, IoT platforms).

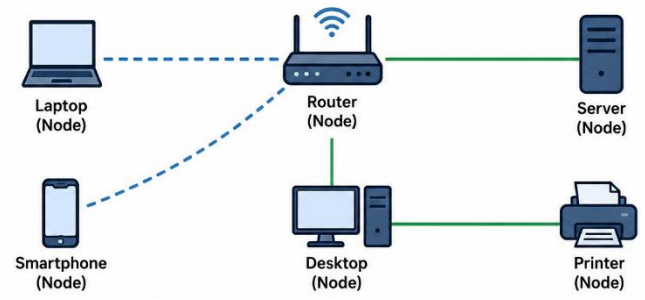


Figure: Network

Basic components of network:

- Nodes: hosts (PCs, servers, sensors), networking devices (switches, routers)
- Links: wired (Ethernet, fiber) or wireless (Wi-Fi, cellular)
- Protocols: rules for communication (e.g., TCP, IP, HTTP)

1.8.2. Types of Networks (by Scale)

- PAN (Personal Area Network): Very small, around a person (Bluetooth, USB).
- LAN (Local Area Network): Building/campus level; high speed, low delay (Ethernet, Wi-Fi in college lab).
- MAN (Metropolitan Area Network): City level (e.g., cable TV network, city Wi-Fi).
- WAN (Wide Area Network): Country/global (Internet, leased lines).

For IoT: sensors → LAN (via gateway) → WAN (cloud).

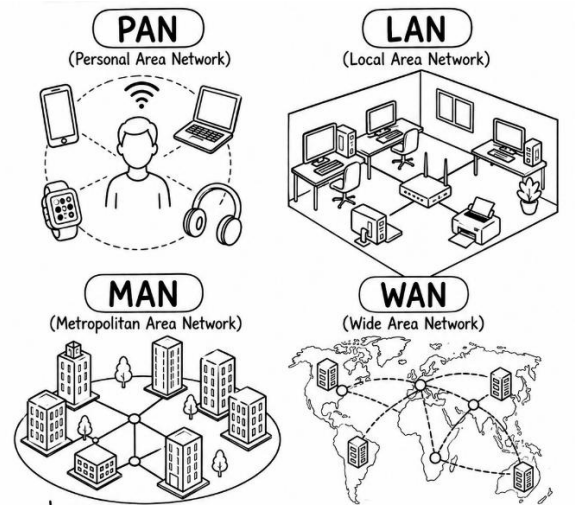


Figure: Types of Networks

1.8.3. Network Topologies

Physical or logical arrangement of nodes and links.

- Bus: All devices on a single cable. Simple, but single point of failure.
- Star: All devices connect to a central hub/switch. Easy to manage; hub/switch failure affects all.
- Ring: Each node connects to two neighbors in a circle. Data travels in one direction; break in ring can stop communication.
- Tree: Hierarchical (like multiple stars connected). Common in campus networks.
- Mesh: Many interconnections; high reliability, more cost. Used in backbone and some IoT mesh networks.

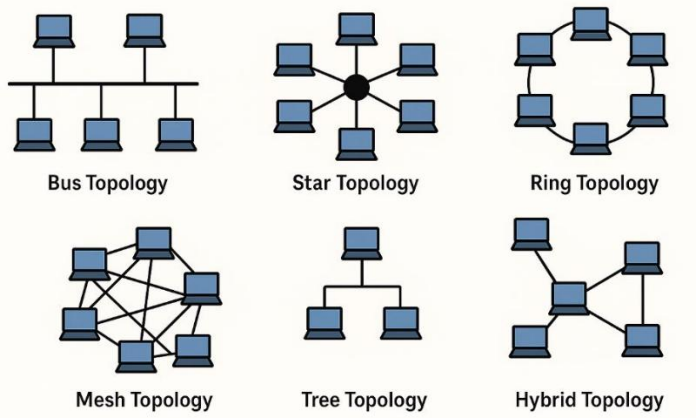


Figure: Network Topologies

1.8.4. Communication Modes

- Simplex: One-way only (e.g., traditional TV broadcast).
- Half-duplex: Both ways, but not at same time (e.g., walkie-talkie).

- Full-duplex: Both ways simultaneously (e.g., phone call, Ethernet).[1]

Data can be sent as:

- Unicast: One sender → one receiver
- Broadcast: One sender → all in network
- Multicast: One sender → selected group

1.8.5. OSI Reference Model (7 Layers)

OSI (Open Systems Interconnection) is a conceptual 7-layer model to understand how data moves from one application to another across a network.

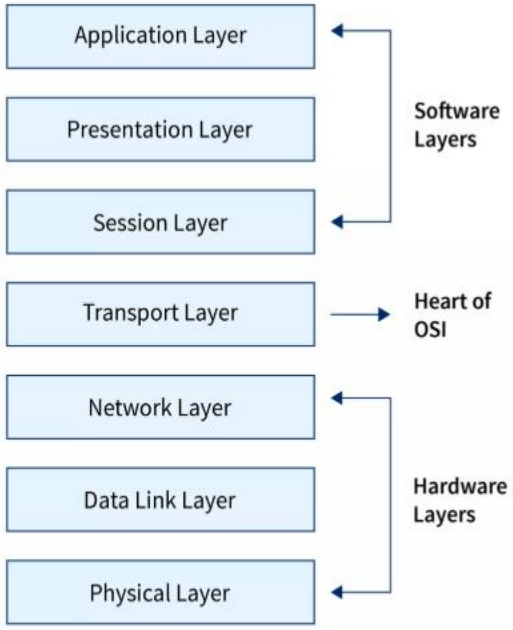


Figure: OSI Reference Model

From bottom to top:

1. **Physical Layer (Layer 1)**
    - Deals with raw bits over media (cables, radio).
    - Defines voltages, connectors, bit rates.
    - Devices: cables, hubs, repeaters.
2. **Data Link Layer (Layer 2)**
    - Groups bits into **frames**.
    - Handles MAC addressing, error detection, flow control on a single link.
    - Protocols: Ethernet, Wi-Fi (802.11), ARP.
    - Devices: switches, bridges.

3. **Network Layer (Layer 3)**
    - Handles **logical addressing** (IP addresses) and **routing** between networks.
    - Data unit: **packet**.
    - Protocols: IP, ICMP, routing protocols (OSPF, BGP).
    - Devices: routers.
4. **Transport Layer (Layer 4)**
    - End-to-end delivery between applications on different hosts.
    - Data unit: **segment** (TCP) / **datagram** (UDP).
    - Protocols: TCP (reliable), UDP (fast, unreliable).
    - Functions: segmentation, reassembly, flow control, error recovery, ports.
5. **Session Layer (Layer 5)**
    - Manages sessions (dialog control) between applications.
    - Establishes, maintains, terminates connections.
6. **Presentation Layer (Layer 6)**
    - Data translation, encryption, compression.
    - Formats: ASCII, JPEG, encryption schemes.
7. **Application Layer (Layer 7)**
    - Network services directly used by applications.
    - Protocols: HTTP, HTTPS, FTP, SMTP, DNS, SSH.

1.8.6. TCP/IP Model (Practical Internet Model)

TCP/IP is the real-world model used on the Internet. It has fewer layers and maps roughly to OSI.

Common 4-layer view:

1. **Link (Network Access) Layer**
    - Combines OSI Physical + Data Link.
    - Ethernet, Wi-Fi, MAC addressing, framing.
2. **Internet Layer**
    - Corresponds to OSI Network layer.
    - IP addressing, routing, fragmentation.
    - Protocols: IP, ICMP, ARP (sometimes placed here or in Link).
3. **Transport Layer**

- Same as OSI Transport.
- TCP, UDP, ports, reliability, flow control.[3][10][13]

4. **Application Layer**
   - Combines OSI Application + Presentation + Session.
   - HTTP, DNS, SMTP, FTP, SSH, etc.

Sometimes a 5-layer "hybrid" model is taught: Physical, Data Link, Network, Transport, Application (used in many B.Tech courses).

1.8.7. Protocols: TCP vs UDP

Both are transport layer protocols.

- **TCP (Transmission Control Protocol)**
  - Connection-oriented (3-way handshake: SYN, SYN-ACK, ACK).
  - Reliable: guarantees in-order delivery, retransmits lost packets.
  - Flow control, congestion control.
  - Used by: HTTP/HTTPS, FTP, SMTP, SSH.
- **UDP (User Datagram Protocol)**
  - Connectionless: no handshake, just send datagrams.
  - Unreliable: no guarantee of delivery/order.
  - Lower overhead, lower latency.
  - Used by: DNS, VoIP, video streaming, many IoT telemetry protocols.

For IoT: UDP is often preferred for small, frequent sensor data; TCP for configuration, firmware updates, critical control.

1.8.8. IP Addressing Basics

IPv4

- 32-bit address, written as 4 decimals: e.g., `192.168.1.10`.
- Divided into **network part** and **host part** using a **subnet mask** (e.g., `255.255.255.0`).
- Classes (old style): A, B, C, D, E; now mostly replaced by **CIDR** (Classless Inter-Domain Routing) like `192.168.1.0/24`.

Special ranges (private):

- `10.0.0.0 – 10.255.255.255`

- 172.16.0.0 – 172.31.255.255
- 192.168.0.0 – 192.168.255.255

IPv6

- 128-bit address, hexadecimal, e.g., 2001:0db8:85a3::8a2e:0370:7334.
- Solves IPv4 exhaustion, simplifies some routing.

NAT (Network Address Translation)

- Allows many private IPs to share one public IP.
- Common in home/college routers.

1.8.9. Network Devices and Their Roles

Important for labs and viva.

- **Repeater**:
  - Physical layer device.
  - Amplifies/regenerates signals to extend distance.
- **Hub**:
  - Physical layer, multi-port repeater.
  - Broadcasts incoming data to all ports (no intelligence).
- **Bridge**:
  - Data Link layer device.
  - Connects two LAN segments; forwards frames based on **MAC addresses**.
  - Filters traffic; reduces collisions.
- **Switch**:
  - Multi-port bridge (Layer 2, sometimes Layer 3).
  - Learns MAC addresses, forwards frames only to the correct port.
  - Supports full-duplex, VLANs, better performance than hub.[8][9]
- **Router**:
  - Network layer device.
  - Connects different networks (LAN ↔ WAN).
  - Uses **IP addresses** and routing tables to forward packets.
  - Performs NAT, DHCP (in home routers).
- **Gateway**:

- Can operate at higher layers (Transport/Session/Application).
- Connects networks using **different protocols** (e.g., IoT protocol to HTTP cloud API).
- More "intelligent" than a router; often a small computer or server.

In IoT:

Sensor → (maybe) microcontroller → switch/hub → **gateway** → router → Internet/cloud.

1.8.10. Basic Network Services

- **DNS (Domain Name System)**
  - Converts domain names (`www.google.com`) to IP addresses.
  - Usually uses UDP port 53.
- **DHCP (Dynamic Host Configuration Protocol)**
  - Automatically assigns IP address, subnet mask, gateway, DNS to devices.
  - Uses UDP.
- **HTTP/HTTPS**
  - Web traffic; HTTP (port 80), HTTPS (port 443, encrypted).
- **FTP, SMTP, SSH**
  - File transfer, email, secure remote login respectively.

1.8.11. Simple Data Flow Example (Browser → Web Server)

Using TCP/IP + OSI ideas:

1. You type `http://example.com` in browser (Application layer).
2. DNS resolves `example.com` to an IP (still Application, uses UDP).
3. Browser opens a TCP connection (Transport: 3-way handshake).
4. HTTP request is created (Application), passed to TCP (segmented), then to IP (packets with source/dest IP).
5. Packets go through your NIC (Data Link: frames with MAC), then Physical layer (bits on cable/Wi-Fi).
6. Router receives frames, extracts packets, checks IP, forwards toward server.
7. Server reverses the process, sends HTTP response back.

1.8.12. Quick Formula / Terms to Remember

- **Bandwidth**: Maximum data rate (bits per second).
- **Throughput**: Actual achieved data rate.
- **Latency**: Time taken for a packet to travel from source to destination.
- **Jitter**: Variation in latency (important for VoIP/video).

Basic relation (conceptual):

$$\text{Time to send a file} \approx \frac{\text{File size (bits)}}{\text{Throughput (bps)}} + \text{Latency}$$

1.9. Communication Protocols in IoT

A communication protocol is a set of rules that lets connected devices share data. It controls how data is formatted, sent, received, and checked for errors. This standard language helps different hardware and software systems talk to each other smoothly.

1.9.1. MQTT (Message Queuing Telemetry Transport)

Type: Application layer, publish-subscribe protocol, runs over TCP/IP.

Key Concepts:

- **Broker** – central server that routes messages
- **Publisher** – device that sends data (publishes to a "topic")
- **Subscriber** – device that receives data (subscribes to a "topic")
- **Topic** – a hierarchical string like home/livingroom/temperature

QoS Levels:

Level	Meaning
0	At most once (fire and forget)
1	At least once (may duplicate)
2	Exactly once (guaranteed, slowest)

Features:

- Lightweight header (as small as 2 bytes)
- Works well on low-bandwidth, unreliable networks
- Persistent sessions and "Last Will and Testament" (LWT) message if a device disconnects unexpectedly
- Retained messages for late subscribers

Use case: Smart home sensors, fleet tracking (Facebook Messenger also used MQTT historically for low-power mobile messaging).

1.9.2. CoAP (Constrained Application Protocol)

Type: Application layer, designed for **constrained devices** (low power, low memory), runs over **UDP**.

Key Concepts:

- REST-like model (GET, POST, PUT, DELETE) — similar to HTTP but lightweight
- Uses **Confirmable (CON)** and **Non-confirmable (NON)** messages to add reliability over UDP
- Supports **Observe** option (like a subscribe mechanism, similar to MQTT topics)
- Small header (~4 bytes)

Features:

- Built for machine-to-machine (M2M) communication
- Can be easily translated/mapped to HTTP via proxies
- Supports multicast (useful for group communication in sensor networks)
- Uses DTLS for security (since it runs on UDP, not TCP)

Use case: Low-power sensor networks, smart lighting systems where devices sleep most of the time.

1.9.3. ZigBee

Type: Not just application layer — it's a **full wireless communication protocol stack** (based on IEEE 802.15.4) covering physical, MAC, network, and application layers.

Key Concepts:

- **Mesh networking** – devices relay data for each other, extending range
- **Node types:**
    - **Coordinator** – one per network, manages the network
    - **Router** – relays data, extends network
    - **End Device** – low-power, communicates only with its parent

Features:

- Very low power consumption (battery can last years)
- Short range (~10–100 m)
- Low data rate (20–250 kbps) — not meant for large data, only small control/sensor packets
- Operates in 2.4 GHz (also 868/915 MHz bands)
- Strong security via AES-128 encryption

Use case: Home automation (smart bulbs, locks), industrial sensor networks — anywhere you need many battery-powered nodes talking over short range.

1.9.4. HTTP (Hypertext Transfer Protocol)

Type: Application layer, **request-response** model, runs over TCP/IP.

Key Concepts:

- Client sends a request (GET, POST, PUT, DELETE), server sends a response
- **Stateless** — each request is independent
- Uses well-defined status codes (200 OK, 404 Not Found, 500 Server Error, etc.)

Features:

- Universally supported, easy to integrate with web services and REST APIs
- Heavier header overhead compared to MQTT/CoAP — not ideal for constrained/battery devices
- Reliable (built on TCP)
- HTTPS variant adds TLS/SSL security

Use case: Cloud dashboards, REST APIs connecting IoT gateways to backend servers, devices with good power/bandwidth availability (like smart cameras, gateways).

Comparison Table

Feature	**MQTT**	**CoAP**	**ZigBee**	**HTTP**
Transport layer	TCP	UDP	IEEE 802.15.4 (own stack)	TCP
Model	Pub/Sub	Request-Response (REST-like)	Mesh networking	Request-Response
Power consumption	Low	Very Low	Very Low	High
Header size	Small (2 bytes)	Small (4 bytes)	Small	Large
Range	Depends on network (internet-wide)	Depends on network	Short (10–100 m)	Internet-wide
Reliability	QoS 0/1/2	CON/NON messages	ACK-based	TCP guarantees
Best for	Real-time sensor data over internet	Constrained device REST APIs	Local mesh	Web-based/cloud integration

Feature	MQTT	CoAP	ZigBee	HTTP
			sensor/actuator networks	

1.10. Sensor Networks: Types, Topologies, and Protocols

Here are structured notes covering this topic for your B.Tech studies.

1.10.1 Introduction to Sensor Networks

A Wireless Sensor Network (WSN) is a collection of spatially distributed, autonomous sensor nodes that monitor physical or environmental conditions (temperature, pressure, sound, humidity, motion, etc.) and cooperatively pass data through the network to a central location called the sink or base station (BS).

Key components of a sensor node:

- Sensing unit (sensor + ADC)
- Processing unit (microcontroller + memory)
- Transceiver unit (communication)
- Power unit (battery, often the limiting factor)

1.10.2. Types of Sensor Networks

A. Based on Deployment Environment

Type	Description	Example Use
Terrestrial WSN	Nodes deployed on land, dense/random placement	Agriculture, forest monitoring
Underground WSN	Nodes buried underground; costly, signal attenuation is high	Mining, soil monitoring
Underwater WSN	Nodes float/submerged in water; use acoustic waves (RF doesn't work well underwater)	Ocean monitoring, pollution control
Multimedia WSN	Nodes with cameras/microphones to capture video, audio, images	Surveillance, traffic monitoring
Mobile WSN	Nodes can move and reposition themselves	Battlefield tracking, robotics

B. Based on Node Capability

- **Homogeneous WSN** – all nodes have same capability (energy, processing).
- **Heterogeneous WSN** – nodes differ in capability (some act as cluster heads with more power/processing).

1.10.3. Sensor Network Topologies

Topology decides how nodes connect and communicate with the base station.

1. Star Topology

- All nodes connect directly to a single central base station.
- **Pros:** Simple, low latency.
- **Cons:** Limited range, single point of failure (BS).

2. Mesh Topology

- Every node can communicate with multiple neighboring nodes; data hops from node to node until it reaches BS.
- **Pros:** High reliability, self-healing (alternate paths if a node fails), extends range.
- **Cons:** Higher power consumption, complex routing.

3. Tree Topology

- Hierarchical structure — a combination of star and mesh; nodes connect to a parent node which connects further up to BS.
- **Pros:** Easy to scale, structured routing.
- **Cons:** If a parent node fails, all its children lose connectivity.

4. Cluster-based (Hierarchical) Topology

- Nodes grouped into clusters, each with a **Cluster Head (CH)** that aggregates and forwards data to BS.
- Used in protocols like **LEACH**.
- **Pros:** Energy efficient, reduces data redundancy via aggregation.
- **Cons:** CH becomes energy bottleneck (needs rotation).

5. Hybrid Topology

- Combines two or more topologies (e.g., star + mesh) to balance reliability and energy efficiency.

1.10.4. Sensor Network Protocols

Protocols are generally studied layer-wise (like OSI model) but with adaptations since WSNs are energy/resource constrained.

A. MAC Layer Protocols (Medium Access Control)

Goal: minimize collisions and conserve energy.

Protocol	Mechanism	Notes
CSMA/CA	Carrier sense before transmission	Contention-based, simple but energy-wasteful (idle listening)
TDMA	Time slots allocated to each node	Contention-free, energy efficient, needs synchronization
S-MAC	Sensor-MAC — nodes sleep/wake periodically	Reduces idle listening, saves energy
T-MAC	Timeout-MAC — adaptive duty cycle	Improves on S-MAC by adjusting sleep dynamically
Hybrid (Z-MAC)	Combines CSMA + TDMA	Balances flexibility and efficiency

B. Routing Protocols (Network Layer)

(i) Flat Routing – all nodes play the same role

- **Flooding:** Every node forwards to all neighbors (simple, high redundancy/energy waste).
- **Gossiping:** Random neighbor selected instead of all (reduces overhead).
- **SPIN (Sensor Protocols for Information via Negotiation):** Uses metadata negotiation to avoid redundant data.
- **Directed Diffusion:** Data-centric; sink sends interest, nodes respond with matching data along reinforced paths.

(ii) Hierarchical Routing – cluster-based, more energy efficient

- **LEACH (Low Energy Adaptive Clustering Hierarchy):** Randomly rotates cluster-head role to balance energy load across nodes.
- **PEGASIS (Power-Efficient Gathering in Sensor Information Systems):** Nodes form a chain; data passed node to node to one leader, reducing overhead vs LEACH.
- **TEEN / APTEEN:** Threshold-based, reactive protocols good for time-critical applications.

(iii) Location-based Routing

- **GPSR (Greedy Perimeter Stateless Routing):** Uses geographic node position to route data (greedy forwarding to node nearest destination).
- **GEAR (Geographic and Energy Aware Routing):** Combines location info with residual energy to choose routes.

C. Transport Layer Protocols

- Ensure reliable data delivery with congestion control while conserving energy.
- Examples: **PSFQ (Pump Slowly Fetch Quickly)**, **CODA (Congestion Detection and Avoidance)**.

D. Application Layer Protocols

- Handle tasks like time synchronization, localization, network management.
- Example: **Sensor Management Protocol (SMP)**, **Task Assignment and Data Advertisement Protocol (TADAP)**.

1.11.Introduction to IoT Security and Privacy Fundamentals.

1.11.1. Why IoT Security Is Different (and Harder)

- **Resource constraints** — Many devices have limited CPU, memory, and battery, making strong encryption/authentication difficult to implement.
- **Heterogeneity** — Devices use different hardware, protocols (MQTT, CoAP, Zigbee), and OS versions — no single security standard fits all.
- **Massive scale** — Billions of connected endpoints increase the attack surface exponentially.
- **Physical exposure** — Devices are often deployed in unsecured, publicly accessible locations (unlike servers in a data center).
- **Long lifecycle, poor patching** — IoT devices often run outdated firmware for years with no update mechanism.
- **Default/weak credentials** — Many devices ship with hardcoded or default passwords.

1.11.2. Common IoT Security Threats and Attacks

Attack Type	Description
Botnets / DDoS	Compromised devices used to flood a target with traffic (e.g., Mirai)
Man-in-the-Middle (MITM)	Attacker intercepts communication between device and server

Attack Type	Description
Firmware Tampering	Malicious code injected into device firmware
Physical Tampering	Direct hardware access to extract keys or reprogram devices
Eavesdropping / Sniffing	Capturing unencrypted data over the network
Spoofing	Impersonating a legitimate device or user
Denial of Service (DoS)	Overwhelming a device to make it unavailable
Side-channel attacks	Extracting secrets via power consumption, timing, etc.

Case Study: Mirai Botnet (2016)

The Mirai malware scanned the internet for IoT devices (routers, IP cameras, DVRs) still using **factory-default usernames/passwords**, infected them, and used the resulting botnet of hundreds of thousands of devices to launch massive DDoS attacks — including one that took down major services like Twitter, Netflix, and Reddit by attacking DNS provider Dyn. It remains the textbook example of why weak default credentials are catastrophic at IoT scale.

1.11.3. Core Security Principles Applied to IoT

The classic **CIA Triad**, adapted for IoT:

- **Confidentiality** — Ensuring sensor/user data isn't exposed to unauthorized parties (encryption in transit and at rest).
- **Integrity** — Ensuring data/firmware isn't altered maliciously (digital signatures, checksums, secure boot).
- **Availability** — Ensuring devices/services remain accessible despite attacks (resilience against DoS/DDoS).

Additional IoT-specific principles:

- **Authentication** — Verifying device and user identity (certificates, mutual TLS, secure tokens).
- **Authorization / Access Control** — Ensuring only permitted entities can perform specific actions (principle of least privilege).
- **Non-repudiation** — Preventing denial of an action performed (audit logs, signed transactions).

1.12.4. Privacy Fundamentals in IoT

IoT devices constantly collect personal, behavioral, and sometimes biometric data (location, health metrics, voice recordings, home occupancy patterns). This raises distinct **privacy** concerns separate from pure security:

- **Data minimization** — Collect only what is strictly necessary.
- **Purpose limitation** — Use data only for the purpose it was collected for.
- **Consent & transparency** — Users should know what data is collected and how it's used.
- **Anonymization / Pseudonymization** — Strip or mask identifiers before storage/sharing.
- **Data ownership & control** — Users should be able to access, export, or delete their data.
- **Third-party data sharing risks** — Many IoT vendors share data with advertisers/analytics firms, often without clear user awareness.

Key distinction for exams: *Security* protects data/systems from unauthorized access or damage; *Privacy* concerns how personal data is collected, used, and shared — even by authorized parties. A system can be fully secure but still violate privacy (e.g., a company legally collecting excessive data while encrypting it perfectly).

1.12.5. Security & Privacy Best Practices (Solutions)

Category	Practice
Device level	Secure boot, hardware root of trust, disable unused ports/services
Authentication	Unique per-device credentials, avoid hardcoded passwords, mutual TLS
Communication	End-to-end encryption (TLS/DTLS), lightweight cryptography for constrained devices
Network	Network segmentation, firewalls, intrusion detection systems (IDS)
Updates	Secure, signed Over-The-Air (OTA) firmware updates
Data handling	Encrypt data at rest, apply data minimization, anonymize where possible
Governance	Follow standards like NIST IoT guidelines, ISO/IEC 27400, and privacy laws (e.g., GDPR)